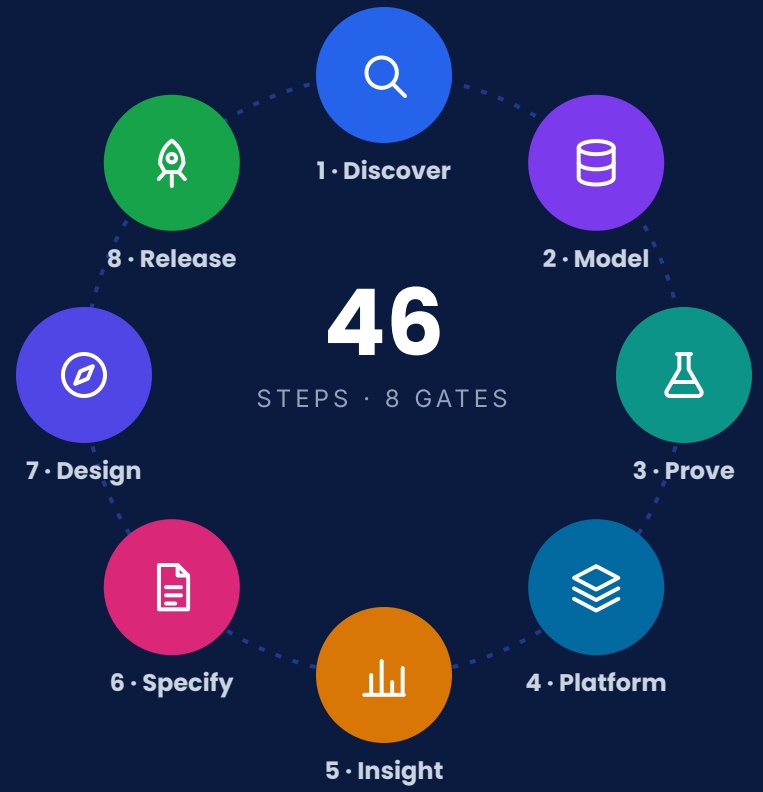
talenciaglobal
Great talent. Global opportunities

DETAILED PLAYBOOK

DATA-DRIVEN DEVELOPMENT

The Data-Driven Development Lifecycle

A step-by-step path from a raw problem statement to a released, measured product. Data first, tests before code, insight before specification.



Talencia Global Version 1.0 · October 2026 For learners, mentors and delivery teams

PURPOSE

Building the app is the easy part now.

AI coding agents can generate an application in hours. What still decides success is the data, the requirements, the features and the proof. This playbook puts those first.

Data

The data model, its rules and realistic data decide whether the product works at all.

Requirements

Most failures trace back to a requirement that was never properly understood.

Feature discovery

Knowing what competitors do, and what they miss, decides what is worth building.

Testing

The first and foremost discipline, and the one most often skipped.

HOW TO USE THIS PLAYBOOK

The steps are common to everyone. The depth, judgement and originality brought to each step is what separates an average outcome from an excellent one. Every step ends with a **Go deeper** prompt for exactly that.

01 Data first
Model the data and prove it with realistic data before designing screens or services.

02 Tests before code
Every rule, workflow and target has a test before any code is generated.

03 Insight before specification
KPIs and executive analytics come before the PRD, so the product is specified around measurable outcomes.

04 Everything traceable
Each requirement links to its entities, rules, tests, code and KPIs.

TRADITIONAL VS DATA-DRIVEN

Same goal. The order is reversed.

DIMENSION	TRADITIONAL SDLC	DATA-DRIVEN DEVELOPMENT
Starting point	Requirements, then screens	✓ Requirements, then entities, rules and data
First model built	UI mock-ups and features	✓ Data model, proven with generated data
Testing starts	After code is written	✓ Step 15, on the first sample data
KPIs defined	After launch, if at all	✓ Before the PRD, from real data structures
AI coding tools	Main source of effort	✓ Final step, fed by verified specs and tests
Defects found	Late, from misread requirements	✓ Early, by workflow coverage and data tests
Definition of done	Works on the happy path	✓ Every rule, workflow, NFR and KPI tested and traceable

TRADITIONAL ORDER

Requirements → Design → Code → Test → Release

Data and testing arrive last, when changes are most expensive.

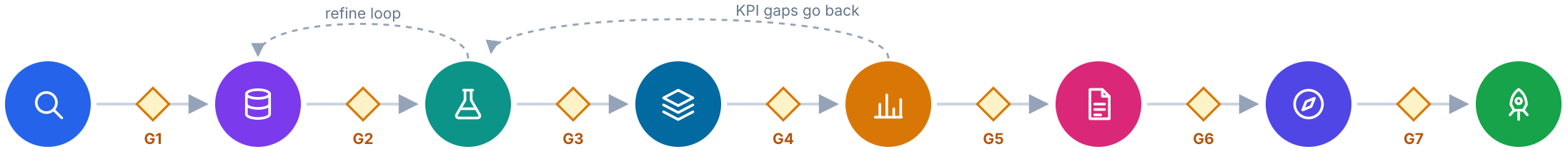
DATA-DRIVEN ORDER

Requirements → Data → Proof → Insight → Spec → Design → Code

The riskiest questions are answered first, when changes are cheapest.

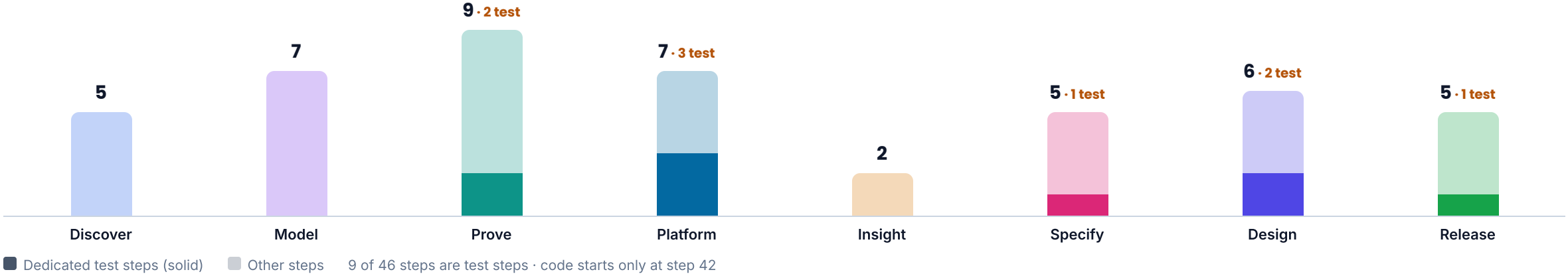
THE LIFECYCLE AT A GLANCE

Eight phases, 46 steps, testing built in throughout



1 · STEPS 1–5	2 · STEPS 6–12	3 · STEPS 13–21	4 · STEPS 22–28	5 · STEPS 29–30	6 · STEPS 31–35	7 · STEPS 36–41	8 · STEPS 42–46
Discover	**Model**	**Prove**	**Platform**	**Insight**	**Specify**	**Design**	**Release**
Understand before you model	Define the data the business runs on	Test the model with realistic data	Load real volume, automate the proof	Turn data into decisions	Write the product down, grounded in proof	Shape the solution, write the tests first	Build, verify, ship and learn

← Step 46 learnings feed step 1 of the next cycle



ANATOMY OF A STEP

Every step reads the same way

EVERY STEP HAS FIVE PARTS

PURPOSE	Why the step exists and what risk it removes.
DO	The core activities.
PRODUCE	The artifact the step must leave behind.
DONE WHEN	The exit test. If it is not met, the step is not complete.
GO DEEPER	Where to add depth and originality beyond the baseline.

EXAMPLE

08 Entity Lifecycle and State Machines

PURPOSE	Define the states each core entity moves through. Most rule defects live here.
DO	List states, allowed transitions, who triggers each and the guarding conditions.
PRODUCE	State diagram and transition table per stateful entity.
DONE WHEN	Every status has a way in and a way out, or is marked final.

GO DEEPER
List illegal transitions explicitly. Each becomes a negative test.

THREE RULES THROUGHOUT

1. Give everything an ID

Requirements, entities, rules, tests and KPIs all get IDs (REQ-, ENT-, BR-, TC-, KPI-) so they can be traced.

2. Keep a decision log

Record every assumption and design choice with its reason. This is where individual thinking becomes visible.

3. Loop back when needed

A gap found later sends the work back to the step that owns it. Moving forward with a known gap is not allowed.

PHASE 1 · DISCOVER

Understand before you model

PHASE 1 OF 8

Discover

Establish the problem, the people, the market, the quality targets and the real workflows. Nothing is modelled yet.

Steps 1–5

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
01 Requirement Analysis Docket	Turn the raw brief into a structured, unambiguous problem statement.	Split the brief into goals, scope, out-of-scope, assumptions and open questions. Give every requirement an ID.	Requirements docket: ID, statement, source, priority, ambiguity notes.	Every sentence of the brief maps to a requirement ID or an explicit out-of-scope line.	Turn vague words like "fast" and "many" into numbers. Rank open questions by design impact.
02 Stakeholders, Personas and Roles	Know who uses the system, who owns it and who is affected.	List every human and system actor with goals, permissions and frequency of use.	Persona cards and a role–permission matrix.	Every requirement names at least one actor.	Include edge actors: auditors, support staff, regulators, scheduled jobs.
03 Feature Discovery	Find what the product must do and what market leaders already do.	Study at least three competing products. Classify features as must-have, should-have or differentiator.	Feature inventory with a competitor coverage matrix.	Every feature traces to a requirement or is flagged as a new insight.	Find what competitors lack and argue why it matters.
04 NFR Discovery	Fix measurable quality targets before the design locks them out.	Set targets for scale, response time, availability, security, privacy, compliance and accessibility.	NFR register: ID, metric, target, how it is measured.	Every NFR has a number and a test method.	Derive targets from volumes, not guesses. Name the regulations that apply, such as the DPDP Act 2023.
05 Business Workflow Analysis	Map how work actually flows from end to end.	For each core workflow record trigger, steps, actors, decisions, exceptions and outputs.	Workflow diagrams and an exception catalogue.	Every must-have feature appears in at least one workflow.	Map cancel, refund, retry and timeout paths, not only the happy path.

Exit · G1 · Problem understood Every requirement has an ID and an actor. Every NFR has a numeric target. Every must-have feature appears in a workflow.

PHASE 2 · MODEL

Define the data the business runs on (1/2)

PHASE 2 OF 8

Model

Turn requirements and workflows into entities, attributes, relationships, rules and a first data dictionary, with privacy designed in.

Steps 6–12 · part 1 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
06 Entities Discovery	Identify everything the business needs to keep data about.	Extract nouns from requirements, features and workflows. Merge synonyms; separate entities from attributes.	Entity catalogue: ID, name, definition, owner.	Every workflow step reads or writes at least one named entity.	Find hidden entities: history, audit trail, configuration, reference data.
07 Attributes Discovery	Describe each entity precisely.	List attributes with type, length, mandatory flag, unit and example values.	Attribute list per entity.	No attribute is missing a type and an example value.	Separate stored from derived values. Flag multi-valued attributes.
08 Entity Lifecycle and State Machines	Define the states each core entity moves through. Most rule defects live here.	List states, allowed transitions, who triggers each and the guarding conditions.	State diagram and transition table per stateful entity.	Every status has a way in and a way out, or is marked final.	List illegal transitions explicitly. Each becomes a negative test.
09 Database, Tables and Relationships Discovery	Turn the entities into a logical database design.	Define keys, cardinality, optionality and junction tables. Normalise to 3NF; denormalise only with a reason.	Logical ER model v0.	Every relationship has cardinality stated in both directions.	Decide how history is kept (snapshots, SCD, audit tables) and justify it.

PHASE 2 · MODEL

Define the data the business runs on (2/2)

PHASE 2 OF 8

Model

Turn requirements and workflows into entities, attributes, relationships, rules and a first data dictionary, with privacy designed in.

Steps 6–12 · part 2 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
10 Data Dictionary v1	Create the single source of truth for what each field means.	Record table, column, type, constraints, business definition, example and owner.	Data dictionary v1.	A newcomer can write a correct query using only the dictionary.	Add a business-friendly name beside each technical name.
11 Data Rules and Constraints Discovery	Define what makes data valid.	Specify keys, uniqueness, not-null, ranges, formats, allowed values and cross-field rules.	Data rules catalogue: ID, rule, table, enforcement point.	Every rule says where it is enforced: database, application or pipeline.	Capture cross-table rules, e.g. invoice total equals the sum of its lines.
12 Data Classification and Privacy	Design security and privacy in, not on.	Classify columns as public, internal, confidential or personal. Set masking, encryption, retention, consent and audit rules.	Data classification register.	Every personal-data column has a masking rule and a retention period.	Design for right to erasure. Test data never holds real personal data.

Exit · G2 · Model defined Dictionary v1 is complete. Every rule states where it is enforced. Every personal-data column is classified with a retention period.

PHASE 3 · PROVE

Test the model with realistic data (1/2)

PHASE 3 OF 8

Prove

Generate a realistic sample dataset and test the model against data rules, business rules, workflows and NFRs until it holds.

Steps 13–21 · part 1 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
13 Data Volumetrics and Distribution Profile	Make generated data behave like real data. Uniform data hides real problems.	Estimate rows per table at launch, 1 year and 3 years. Record growth, cardinality, null rates and skew.	Volumetric and distribution sheet.	Every table has volume and distribution assumptions with reasoning.	Model seasonality, peak days and burst traffic.
14 Sample Data Generation (Discovery)	Use a small dataset to expose design flaws early.	Generate rule-following data that matches the profile. Include edge cases and invalid rows on purpose.	Sample dataset and a seeded, reproducible generator.	The data covers every state, relationship and rule.	Build a golden dataset whose query answers are known in advance.
15 Test Suite v1 · Data TEST STEP	Prove data integrity before anything is built on it.	Test keys, orphans, constraints, duplicates, formats and ranges.	Data test suite v1 with expected results and rule IDs.	Every rule has at least one positive and one negative test.	Prove invalid records are rejected, not only that valid ones pass.
16 Schema Coverage against Business Workflow	Check the model supports how the business actually works.	Walk every workflow step against the schema: can it be stored, retrieved and audited?	Coverage matrix (workflow step × table) and gap list.	100% of steps are covered, or each gap is logged with an owner.	Run each workflow as SQL against the sample data.
17 Design-Based Refinement	Close the gaps found by testing and coverage.	Add, split, merge or remove tables and columns. Re-run suite v1 after every change.	Refined model and a change log with reasons.	All gaps are closed, or deferred with a stated reason.	Compare two designs for the hardest area and record the choice.

PHASE 3 · PROVE

Test the model with realistic data (2/2)

PHASE 3 OF 8

Prove

Generate a realistic sample dataset and test the model against data rules, business rules, workflows and NFRs until it holds.

Steps 13–21 · part 2 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
18 Business Rules and Data Constraints v2	Add the business layer on top of the data rules.	Capture policies, calculations, eligibility, limits and approvals. Map each to its enforcement point.	Unified rules catalogue v2.	Every business rule has an ID, owner, enforcement point and planned test.	Express complex rules as decision tables so no combination is missed.
19 Data Dictionary v2	Bring the dictionary in line with the refined model.	Regenerate from the refined schema; add rule IDs, classification and volumetrics.	Data dictionary v2.	A script confirms zero drift between dictionary and schema.	Generate the dictionary from the database so it can never drift.
20 Test Suite v2 · Data, Business and NFR TEST STEP	Extend testing to business rules and quality targets.	Add tests for every business rule and transition, plus NFR checks at target volume.	Test suite v2, every test linked to a rule or NFR ID.	Every rule, transition and NFR has at least one test.	Add boundary tests at the exact limits.
21 Baseline ER and Physical Model	Freeze a proven model the rest of the lifecycle builds on.	Publish the ER model. Add indexes, partitioning, DDL constraints and versioned migrations.	ER diagram v1, DDL scripts, migration v1.	DDL runs cleanly on an empty database and suite v2 passes.	It is a baseline, not final: later changes go through the change log.

Exit · G3 · Model proven Workflow coverage is 100%. Test suite v2 passes on sample data. The DDL runs cleanly and the baseline ER model is published.

PHASE 4 · PLATFORM

Load real volume, automate the proof (1/2)

PHASE 4 OF 8

Platform

Connect the outside world, load full-volume data and automate every test so it runs on demand.

Steps 22–28 · part 1 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
22 Integration and Source Mapping	Define what data enters and leaves, and who owns it at each boundary.	List external systems and sources. Define API or file contracts and map source to target fields.	Integration catalogue and source-to-target mapping.	Every inbound and outbound field is mapped with an owner and a rule.	Define behaviour when a source is late, duplicated or malformed.
23 Data Generation at Scale	Test the system at the volume it must handle.	Generate full-volume synthetic data following the step 13 profile, integrity intact.	Seeded generator and the full dataset.	Volumes and distributions match the profile within tolerance.	Also generate the 3-year volume to find where performance degrades.
24 Data Seeding	Give every environment the same, known starting state.	Load reference, master and test data with re-runnable scripts.	Seeding scripts per environment.	Running the seed twice produces exactly the same state.	Separate seed sets for development, testing and demonstration.
25 Data Ingestion	Load data without ever silently losing or corrupting a record.	Build validate, transform, load. Quarantine rejected rows with a reason; reconcile counts.	Ingestion pipeline and reconciliation report.	Source = loaded + quarantined, and every quarantined row has a reason.	Make the pipeline re-runnable; handle late and corrected records.

PHASE 4 · PLATFORM

Load real volume, automate the proof (2/2)

PHASE 4 OF 8

Platform

Connect the outside world, load full-volume data and automate every test so it runs on demand.

Steps 22–28 · part 2 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
26 Test Automation · Data Quality and Analytics TEST STEP	Make data quality checks repeatable.	Automate completeness, accuracy, consistency, timeliness, uniqueness and validity checks.	Automated data quality suite.	The suite runs with one command and in CI.	Publish a data quality score per table and track it.
27 Test Automation · Business Rules TEST STEP	Prove every business rule automatically, every time.	Automate the rule and state-transition tests from suite v2.	Automated rule suite linked to rule IDs.	Every BR- rule has a passing automated test.	Generate test cases directly from the decision tables.
28 Test Automation · NFR TEST STEP	Prove the quality targets at real volume.	Automate load, performance, concurrency, backup and restore tests, and basic security scans.	NFR suite and a performance baseline report.	Every step 4 target is measured and met, or logged as a gap.	Stress past the target to find the breaking point.

Exit · G4 · Data platform ready Full-volume data is loaded and reconciled. All automated suites pass in CI. NFR targets are met at volume.

PHASE 5 · INSIGHT

Turn data into decisions

PHASE 5 OF 8

Insight

Prove the data can answer the questions leadership will ask, before the product is specified.

Steps 29–30

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
29 KPI Discovery	Define how success is measured, using data the system holds.	Define KPIs per persona: formula, source tables, grain, target, owner, refresh. Separate leading from lagging.	KPI catalogue.	Every KPI is computable from the schema, or the gap goes back to Phase 3.	Build a KPI tree linking operational metrics to business outcomes.
30 Business Intelligence and Analytics (C-Suite)	Show leadership what the data can tell them.	Build an executive dashboard answering the top leadership questions, with drill-downs.	Executive dashboard and an insight narrative.	Every number reconciles with a query on the source tables.	Add anomaly flags, trend explanations and what-if views.

Exit · G5 · Insight validated Every KPI is computable from the schema, and every dashboard number reconciles with its source.

PHASE 6 · SPECIFY

Write the product down, grounded in proof

PHASE 6 OF 8

Specify

Turn everything learned into a product specification that is traceable and testable.

Steps 31–35

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
31 Feature Analysis and Requirements Engineering	Refine features using what the data, rules and KPIs revealed.	Write user stories with acceptance criteria; prioritise with MoSCoW or RICE.	Prioritised backlog of user stories.	Every story has testable acceptance criteria and a requirement ID.	Write criteria as Given / When / Then so they become tests directly.
32 PRD · Product Requirements Document	Bring the whole product into one agreed document.	Consolidate vision, personas, features, NFRs, release scope and KPIs as success metrics.	The PRD.	Every feature has acceptance criteria and a success metric.	Add a "not doing" section with reasons to protect scope.
33 Requirements vs Database Gap Analysis	Confirm the data model supports every requirement, and nothing more.	Map each story to its tables and columns. Find unsupported stories and unused tables.	Gap analysis report.	No unexplained gaps remain: each is fixed or logged.	Quantify each gap's impact on the affected KPIs.
34 Traceability Matrix	Connect everything so the impact of any change is visible.	Link each requirement to features, entities, rules and tests; code and KPIs follow later.	Traceability matrix.	No requirement without a test, no test without a requirement.	Generate the matrix from artifact IDs, never by hand.
35 UAT Scenarios Test Suite Discovery TEST STEP	Define how business users will accept the product.	Write UAT scenarios per persona and workflow in business language, including exception paths.	UAT scenario suite.	Every must-have story and workflow has a UAT scenario.	Use the golden dataset so expected results are known.

Exit · G6 · Specification signed off The PRD is approved. The traceability matrix has no unexplained gaps. The UAT suite is approved.

PHASE 7 · DESIGN

Shape the solution, write the tests first (1/2)

PHASE 7 OF 8

Design

Design from the outside in and complete every test suite before a line of code exists.

Steps 36–41 · part 1 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
36 Prototyping	Let users see and react to the key journeys early.	Build a clickable prototype of the main journeys using the generated data; validate with personas.	Prototype and feedback log.	Every must-have journey is reviewed and feedback logged by story ID.	Show realistic edge cases: long names, empty states, large lists.
37 Architecture Definition	Choose a structure that can meet the NFRs.	Choose style, stack, deployment and security architecture. Record decisions as ADRs.	Architecture definition and ADRs.	Every NFR maps to the choice that satisfies it.	Record rejected options and the trade-offs behind each choice.
38 HLD · High-Level Design	Define the major building blocks and how they interact.	Define components, responsibilities, interfaces, data flows and integration points.	HLD with component and data-flow diagrams.	Every story has a component; every step 22 integration has an interface.	Show behaviour under failure: retries, timeouts, fallbacks.

PHASE 7 · DESIGN

Shape the solution, write the tests first (2/2)

PHASE 7 OF 8

Design

Design from the outside in and complete every test suite before a line of code exists.

Steps 36–41 · part 2 of 2

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
39 LLD · Low-Level Design	Give enough detail for code to be generated without guessing.	Specify modules, API contracts (OpenAPI), sequences, validation, errors and data access.	LLD and API specifications.	Every API states request, response, errors and the rule IDs it enforces.	Define idempotency, versioning and pagination for every API.
40 Test Suite · Functional TEST STEP	Define how each feature is proven to work.	Write functional cases from acceptance criteria and the LLD, with negative and boundary cases.	Functional test suite linked to story IDs.	Every acceptance criterion has a functional test.	Use equivalence partitioning and boundary analysis.
41 Test Suite · Integration, End-to-End and Security TEST STEP	Prove the parts work together and the system is safe.	Write contract, integration, end-to-end and OWASP Top 10 security tests.	Integration, E2E and security suites.	Every workflow has an E2E test; every interface a contract test.	Test access-control abuse, such as one user reading another's data.

Exit · G7 · Design approved Every NFR maps to an ADR. The LLD is complete. All test suites exist before code generation starts.

PHASE 8 · RELEASE

Build, verify, ship and learn

PHASE 8 OF 8

Release

Generate the code, check what the AI produced, run every suite, release safely and feed learnings back.

Steps 42–46

STEP	PURPOSE	DO	PRODUCE	DONE WHEN	GO DEEPER
42 Code Generation	Build quickly from verified inputs.	Generate code with AI agents using LLD, API specs, DDL, rules and test suites as context.	Source code in version control, commits linked to story IDs.	Every in-scope story has code, and the build is clean.	Hand the agent the tests and ask it to make them pass.
43 AI Output Review and Code Verification	Catch what AI agents get wrong: invented fields, skipped rules, insecure code.	Review code against the dictionary, rules and LLD. Run static, dependency and secret scans.	Review log of findings and fixes.	No schema drift, every rule enforced, no high-severity findings.	Log the agent's error patterns and improve prompts for the next cycle.
44 Test Execution and Defect Loop TEST STEP	Prove the built product meets everything specified.	Run every suite. Log defects by severity, fix, re-run regression.	Execution, defect and coverage reports.	No open critical or high defects; every requirement has a passing test.	Trace each defect to the step that should have caught it.
45 Release	Ship safely, and be able to undo it.	Set up CI/CD. Prepare release notes, migration plan, go/no-go checklist and rollback plan.	Release package and signed go/no-go record.	Rollback is rehearsed and every go/no-go item is green.	Use staged rollouts or feature flags to limit impact.
46 Observe and Learn	Confirm value in production and improve the next cycle.	Monitor NFRs and KPIs live, track incidents, run a retrospective.	Monitoring dashboards and a retrospective with actions.	Live KPIs are visible against targets; actions feed step 1.	Compare step 13 and 29 assumptions with production reality.

Exit · G8 · Release ready All suites pass. No critical or high defects are open. The rollback has been rehearsed.

QUALITY GATES

A phase is complete only when its gate passes

G1 Problem understood

Every requirement has an ID and an actor. Every NFR has a numeric target. Every must-have feature appears in a workflow.

G2 Model defined

Dictionary v1 is complete. Every rule states where it is enforced. Every personal-data column is classified with a retention period.

G3 Model proven

Workflow coverage is 100%. Test suite v2 passes on sample data. The DDL runs cleanly and the baseline ER model is published.

G4 Data platform ready

Full-volume data is loaded and reconciled. All automated suites pass in CI. NFR targets are met at volume.

G5 Insight validated

Every KPI is computable from the schema, and every dashboard number reconciles with its source.

G6 Specification signed off

The PRD is approved. The traceability matrix has no unexplained gaps. The UAT suite is approved.

G7 Design approved

Every NFR maps to an ADR. The LLD is complete. All test suites exist before code generation starts.

G8 Release ready

All suites pass. No critical or high defects are open. The rollback has been rehearsed.

Who checks a gate?

A reviewer who did not produce the work: a mentor, a peer or a lead. Work never grades itself.

What if it fails?

The work goes back to the step that owns the gap, not forward with a note attached.

What is the evidence?

The artifacts named in each step's Produce line, with IDs that trace to the requirement.

THE TRACEABILITY THREAD

One ID thread runs from requirement to live KPI

Requirement
REQ- · step 1
What the business needs

→

Feature / story
FEAT- / US- · 3, 31
How a user gets it

→

Entity / table
ENT- · steps 6, 9
The data it depends on

→

Rule
BR- · steps 11, 18
What must always be true

→

Test
TC- · steps 15–41
The proof that it works

→

Code
Commit · step 42
The implementation

→

KPI
KPI- · steps 29, 46
Proof of value in production

The KPI closes the loop back to the requirement it was written for

LIVING ARTIFACTS · CREATED ONCE, UPDATED AS THE LIFECYCLE MOVES

ARTIFACT	CREATED AT STEP	UPDATED AT STEPS
Requirements docket	1	31, 32, 33
Decision log	1	Every step
ER model (logical / physical)	9	17, 21
Data dictionary	10	19, 21
Rules catalogue	11	18, 27
Test suites	15	20, 26–28, 35, 40, 41, 44
Traceability matrix	34	42, 44, 46

WHY IT MATTERS

With one ID thread, any change shows exactly what it affects: which tables, which rules, which tests and which KPIs.

A requirement is finished only when its KPI shows the intended outcome in production.

ANTI-PATTERNS TO AVOID

Nine common ways projects fail, and the step that stops each

ANTI-PATTERN	WHY IT FAILS	PREVENTED BY
✕ **Jumping straight to code with an AI agent**	Code is built on guesses; every missing rule becomes a defect	Steps 1–41 first; code at 42
✕ **Uniform random test data**	Hides skew, peaks and edge cases until production	Steps 13, 14, 23
✕ **Happy-path-only workflows**	Cancellations, retries and failures break the system	Steps 5, 8
✕ **Freezing the ER model too early**	Later KPI or requirement gaps cannot be absorbed	Step 21 baseline + change log
✕ **Privacy added at the end**	Personal data leaks into logs, test data and exports	Step 12
✕ **Tests that only prove valid data passes**	Invalid data is never shown to be rejected	Steps 15, 20
✕ **KPIs defined after launch**	The schema cannot compute them	Step 29, before the PRD
✕ **Trusting AI-generated code unreviewed**	Invented fields and insecure code reach production	Step 43
✕ **Copying someone else's output**	Loses the individual depth that makes work valuable	Go deeper + decision log

THE SUCCESS MANTRA

The steps are shared. The depth is yours.

Follow the sequence, pass every gate, and bring your own research and judgement to each step.

Discover
Understand the problem, the people and the targets before you model anything.

Model
Know your data, its rules and its sensitivity.

Prove
Let realistic data break your model before users do.

Platform
Load real volume, reconcile every record, automate every test.

Insight
If you cannot measure it, the schema is not finished.

Specify
Write it down, link it up, leave no gaps.

Design
Every test exists before the first line of code.

Release
Verify what the AI built, ship what you can roll back, learn from production.